SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

RESA (Real-time Economic Strategy Advisor) is an AI-powered decision-support platform designed to assist Malaysian small business owners in making better strategic decisions. It addresses the lack of accessible, real-time business consultancy by providing personalized, actionable recommendations based on user input and market context.

1.1. Purpose:

This document outlines the system architecture, data flow, and technical design decisions behind the development of RESA. It serves as a reference for developers and evaluators to understand how the system operates and integrates AI into its workflow.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *RESA follows a client-server architecture with AI as a service layer. The system is modular, separating frontend, backend, and AI processing components.*
2. ***Data Flows:***
    a. *User inputs (product details, cost, situation) are processed through backend APIs, combined with contextual logic, and sent to the AI model. The generated strategy is then returned and displayed to the user.*
3. ***Model Process:***
    The system processes inputs in stages:
    a. *Input collection*
    b. *Context structuring*
    c. *AI inference*
    d. *Output formatting into actionable steps*

4. *Role of Reference:*
    a. *This document provides a high-level overview of how RESA is structured and how each component interacts.*

*1.2. **Background**:*

Small online business owners often lack structured tools to analyze their business performance. Decisions are typically reactive and based on intuition, which limits growth potential.

1. *Previous Version:*
    Currently, there is no structured system like RESA being used by target users. Most rely on:
    a. *Manual decision-making*
    b. *Social media trends observation*
    c. *Trial-and-error strategies*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Small Business Owners	Small Business Owners	Small Business Owners
Development Team	Build and maintain system	Clean architecture and scalable design
QA Team	Validate system behavior	Defined inputs, outputs, and edge cases

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based Application*
Architecture	Client-Server with AI Service Integration

RESA consists of three main components:

- **Frontend (User Interface)**
- **Backend API Layer**
- **AI Service Layer (LLM Integration)**

Flow:
User → Frontend → Backend → AI Model → Backend → Frontend

2.1.2 LLM as Service Layer

RESA integrates the AI model as a **core reasoning engine**, not just a feature.

Prompt Construction:

- Structured template combining:
  - Product info
  - Cost structure
  - Business situation
- Example format:

```
Analyze this business:
Product: X
Cost: Y
Situation: Slow sales
Generate actionable strategy.
```

Context Window Handling:

- Input is cleaned and structured before sending
- Maximum token limit enforced (~1000–1500 tokens)
- Excess input is truncated or summarized

© UMHackathon 2026

Response Handling:

- AI output is parsed into:
  - Pricing strategy
  - Promotion ideas
  - Content suggestions
- Reformatted into user-friendly steps

System Interaction:

- Frontend sends request → Backend processes → AI API call
- Backend validates and returns structured output

2.2.2. Sequence Diagram

a. User enters business details
b. Frontend sends request to backend
c. Backend formats prompt
d. Backend calls AI API
e. AI returns generated strategy
f. Backend processes response
g. Frontend displays results

2.2. Technological Stack

2.2.1. Frontend Flask
2.2.2. Backend Node.js / FastAPI
2.2.3. Database PostgreSQL
2.2.4. Cloud/Deployment to be deployed before final round

2.3. Key Data Flows

2.3.1. User inputs business data
Backend validates input
Data is structured into prompt
Prompt sent to AI API
AI generates response
Response formatted and returned

2.3.1.1. Data Flow Diagram (DFD)

a. Entities:
i. User
ii. Backend System
iii. AI API
iv. Database (optional)

b. Processes:
i. Input Processing
ii. AI Inference

© UMHackathon 2026

iii. *Output Formatting*

c. **Normalized Database Schema**

Tables:

- Users (optional)
- Sessions (store previous inputs & outputs)

Normalization: Basic normalization applied to avoid redundancy.

3. *Functional Requirements & Scope*

3.1. *Minimum Viable Product:*

3.1.1. *MVP:*

#	Feature	Description
1	Business Input System	Users enter product, cost, and situation
2	AI Strategy Generator	Generates Tailored business strategy
3	Actionable Output	Provides clear steps to follow

3.2. *Non-Functional Requirements (NFRs)*

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Scalability	Handle multiple users	Cloud deployment
Reliability	Consistent output	Input validation + fallback
Maintainability	Modular codebase	Separate frontend/backend
Token Latency	< 3s response time	Optimized prompts
Cost Efficiency	Minimize token usage	Input truncation + caching

© UMHackathon 2026

3.3. *Out of Scope / Future Enhancements*

a. *Integration with Shopee/TikTok APIs*
b. *Real-time competitor analysis*
c. *Automated posting or pricing updates*
d. *Advanced analytics dashboard*

4. *Monitor, Evaluation, Assumptions & Dependencies*

4.1. *Technical Evaluation:*

4.1.1. ***A/B Testing**: Test different prompt formats to improve output quality*

4.1.2. ***Rollback Strategy**: Fallback to previous prompt version if output quality drops*

4.1.3. ***Priority Matrix:***

Priority	*Condition*	*Action*
P1 Critical	*AI Fails or gives wrong output frequently*	*Fix prompt/fallback*
P2 Medium	*Slow response time*	*Optimize API calls*
P3 Low	*UI Issues*	*Minor Affixes*

4.2. *Monitoring:*

4.2.1. *Track response time*
4.2.2. *Track success/failure rate of AI outputs*
4.2.3. *Monitor API usage*

4.3. *Assumptions:*

a. *Users provide accurate data*
b. *Users have internet access*
c. *AI API is available and stable*

4.4. *External Dependencies:* *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*Core AI Reasoning*	*High – API failure affects system*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Cloud Hosting	*Deployment*	*Medium – downtime risk*

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*

- Day 1–2: Planning & idea validation
- Day 3–5: Development (Frontend + Backend)
- Day 7–8: AI integration & testing
- Day 9: Debugging & polishing
- Day 10: Deliverables & Final submission

5.2. *Team Members Role:*

- Faeyza Ashira – UI & UX
- Ilham Setiabudi – API & logic
- Yuda Sulthan – Prompt design & integration
- Alfiando Athallah – Testing & validation

5.3. *Recommendations:*

- Implement caching to reduce AI costs
- Improve prompt engineering for better accuracy
- Add analytics for user insights

© UMHackathon 2026

Email: umhackathon@um.edu.my